

Solaris: A WebAssembly/WebGL Architecture for Real-Time Viscoelastic Continuum Simulation and Tidal-Force Visualization in the Browser

Magda Hamilton

Velvet Disruption

New York, NY, USA

magda@velvetdisruption.com

Third edition · 24 July 2026 · DOI 10.5281/zenodo.21540788 · CC BY 4.0

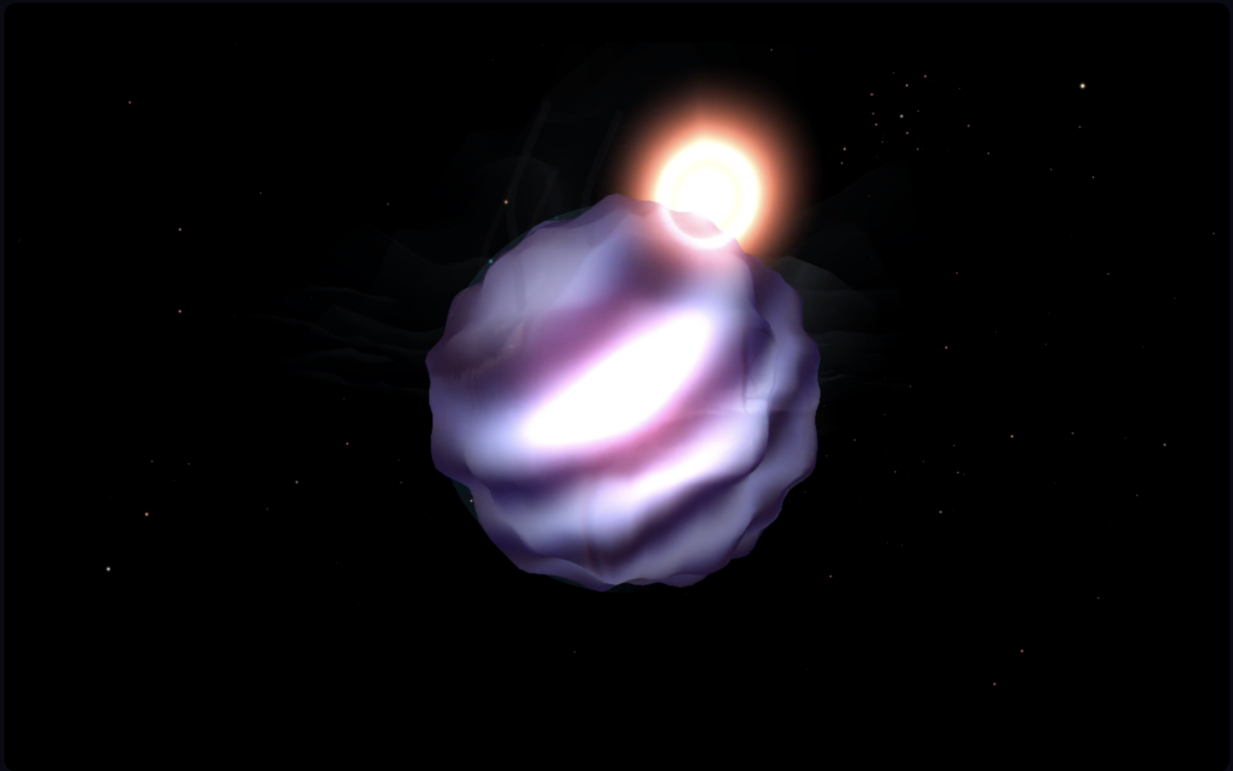


Figure 1. The ocean under the red giant.

ABSTRACT

Solaris is a browser-based, real-time simulation of a planet-sized sentient ocean, built to run entirely client-side through a C++ physics core compiled to WebAssembly and rendered in WebGL. Rather than modeling the surface as water, the ocean is a Kelvin-Voigt viscoelastic gel integrated on a 128×128 grid at 60 Hz, extended with two non-standard fields: a slow memory term that lets deformation persist, and an intent term that drives motion with no external cause. A dual-sun configuration feeds a per-cell tidal potential each frame; a small

fraction displaces the surface while the remainder accumulates as a stress value surfaced in the fragment shader as a subsurface glow, presenting gravitational load as visible effort. Cursor interaction is routed through a drifting mood state that reshapes the material response, and six formation types emerge and dissolve on independent schedules. This report documents the architecture, the physical model, and the rendering and interaction pipeline.

Keywords: viscoelastic simulation; Kelvin-Voigt model; WebAssembly; WebGL; GLSL; real-time physics; tidal forces; three-body problem; interactive art; generative systems; Stanisław Lem

01 — CONCEPT

The Concept

In Stanisław Lem's 1961 novel *Solaris* [1], scientists discover a planet-sized Ocean that thinks — but operates on logic so alien that despite decades of research, humans cannot communicate with it or understand its purpose. It generates elaborate structures, materializes visitors' deepest memories, and maintains a stable orbit around two suns through incomprehensible means.

This project brings that entity to life: a living simulation driven by C++ physics running in WebAssembly. Beneath the visual surface, a Kelvin-Voigt viscoelastic solver models the Ocean as sentient gel — not water — that stores stress, remembers past deformations, and actively shapes its own surface. The Ocean responds to observation, but unpredictably. It creates formations that emerge and dissolve on their own timeline. It appears to perform planetary-scale work we can only glimpse through surface shimmer.

You're studying it. But it's also studying you.

The usual “alien” in games and digital media is a familiar system in a strange skin. *Solaris* is after something harder: a mind that runs on genuinely different logic. The Ocean doesn't perform for you — it observes you back, with the same detached curiosity you bring to it.

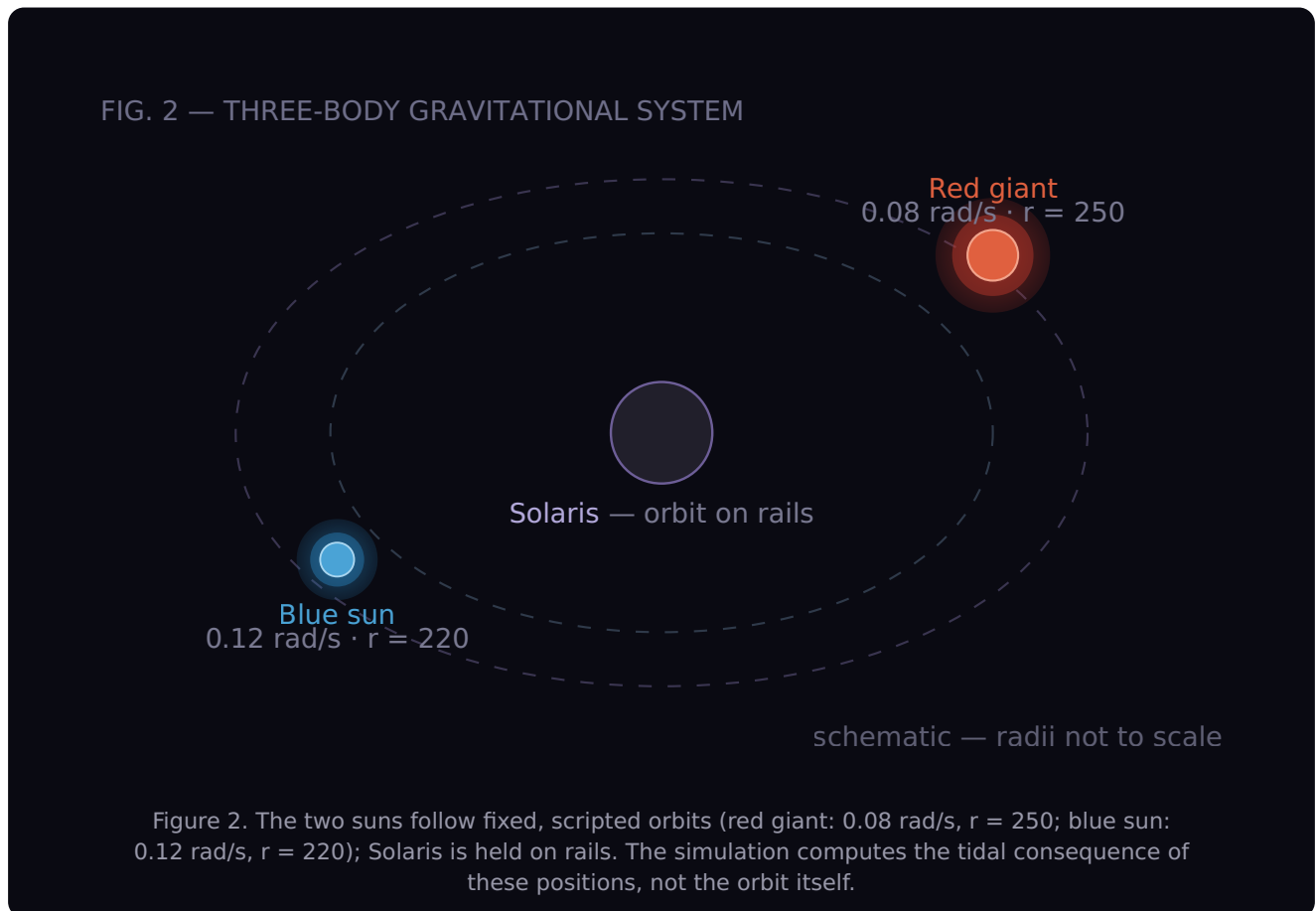
02 — PHYSICS

Solving the Three-Body Problem

One of the most fascinating aspects of Lem's novel: the planet orbits two suns — a configuration that should be gravitationally chaotic. In 1687, Newton formulated universal gravitation [2] and could predict the motion of two celestial bodies perfectly. Then he tried to add a third body. The math disintegrated.

For two centuries, mathematicians sought a general solution. Euler found three collinear equilibrium points; Lagrange added two more, at the vertices of equilateral triangles, and only that second pair is stable — and then only under a mass-ratio condition. Then, by 1890, Henri Poincaré ended the search for a closed-form answer. He showed that the tiniest change in the starting positions could send the trajectories off in completely different directions, folding into structures — the “homoclinic tangles” — too intricate for any tidy formula to reach. It was the first mathematical encounter with what we now call chaos [3, 4].

But Lem imagined something extraordinary: in the novel, the Ocean holds its own orbit through continuous mass redistribution — not solving the equations but rewriting them, treating the three-body problem not as a mathematical puzzle requiring an elegant solution, but as a control problem demanding constant intervention. The simulation doesn't reproduce that feat — the orbit here is on rails — but it stages what the effort would look like, as the next section describes.



Computing the Tidal Field

The two suns move on fixed, scripted paths — the red giant slowly (0.08 rad/s) along a 250-unit radius, the blue sun (a hot main-sequence star, not the hypothetical object the term "blue dwarf" names) faster at 0.12 rad/s and 220 units. Their orbits are on rails: scripted motion, not a gravity simulation.

What the simulation computes each frame is not the orbit but its *consequence*: the tidal field the two suns impose on the surface. Both sun positions feed the C++/WebAssembly solver, which evaluates the tidal potential per cell across the 128×128 grid using the standard formula:

C++ / WASM — TIDAL FORCE

```
// C++ WASM — tidal potential per cell
float tidal = (3.0f * dot * dot - 1.0f) * 0.5f * sun_tidal;

// Only 5% of the tidal term reaches the surface as visible displacement
float leaked_force = total_tidal * 0.05f;
cell.velocity += leaked_force * dt;

// The other 95% never touches the orbit. It's a fiction: a stand-in for
// the labor such an Ocean would spend resisting that pull, accumulated
// only as a stress value that drives the glow.
float counteraction_effort = fabsf(total_tidal) * 0.95f;
cell.stress += counteraction_effort * dt * 0.5f;
```

The trailing `0.5f` is an accumulation rate, not part of the split: it slows how fast stress builds so the glow rises over seconds rather than snapping on. That value is written into a texture the fragment shader reads back as a warm glow beneath the surface, brightest where the load runs heaviest. It isn't decorative — the solver recomputes it sixty times a second. What it is designed to produce is the impression of the Ocean straining to hold itself against two suns, its effort surfacing as light.

Visualizing the Effort

The real challenge was making that invisible effort legible. In the fragment shader I mark “engineering zones” — regions that brighten where the tidal load runs highest. When the two suns align, pulsing FBM patterns rise across the surface:

GLSL — ENGINEERING ZONES

```
float redEngineering = calculateEngineeringZone(uRedSunPos, uRedSunColor, 0);
float blueEngineering = calculateEngineeringZone(uBlueSunPos, uBlueSunColor, 1);
```

Where the two zones overlap they blend toward purple — the signature of both tidal fields loading the same patch of surface at once, their interference rendered as color rather than resolved into any actual counter-force.

03 — RENDERING

The Living Surface

The Ocean's visual surface is driven by two systems working together. Custom vertex and fragment shaders handle the rendering — multi-layered wave displacement with Fractal Brownian Motion creates a surface that never repeats. But underneath, a C++/WebAssembly simulation computes the actual physics: a 128×128 grid of cells, each tracking height, velocity, stress, strain, viscosity, elasticity, intent, and memory.

Viscoelastic Physics (C++/WASM)

The material under the Ocean's surface isn't water — it's a Kelvin-Voigt viscoelastic gel. Water in real-time graphics follows the wave equation ($\partial^2 h / \partial t^2 = c^2 \nabla^2 h$); the gel obeys a different constitutive law, in which each cell's stress is the sum of an elastic term set by how far it is

displaced and a viscous term set by how fast it is moving. In practice the surface pushes back against being deformed, holds the load for a beat, and lets go in a damped wobble instead of a clean travelling wave. The scheme is lumped rather than continuum: each cell is a damped spring integrated explicitly, with a nearest-neighbour coupling term standing in for spatial stress transport. It approximates the Kelvin-Voigt response on a grid; it does not solve the continuum equations.

C++ — KELVIN-VOIGT VISCOELASTIC SOLVER

```
float elastic_force = -elasticity * (height - rest_position);
float viscous_force = -viscosity * velocity;
float neighbor_pull = (avg_neighbor_height - height) * stress_propagation;

// Intent — the Ocean reaches toward a target height it sets for itself
float intent_force = 0.0f;
if (intent_strength > 0.01f) {
    intent_force = (intent - height) * intent_strength * intent_responsiveness;
    intent_strength *= (1.0f - dt * 0.5f); // the reach decays — it changes its mind
}

float total_force = elastic_force + viscous_force + neighbor_pull + intent_force;
velocity += total_force * dt;
velocity *= damping;
height += velocity * dt;
```

The intent field is what tips the material from merely odd into alien. Each cell reaches toward a height target it sets for itself, and because that reach decays frame by frame, the Ocean commits to a direction and then abandons it — the surface moves with no external cause and no fixed goal. Memory runs the other way: the equilibrium each cell settles toward is a slow average of how it has recently been pushed around — a leaky integrator over its deformation history with a time constant of roughly thirty seconds — so the surface never quite comes back to the shape it started from.

The Ambient Field

Intent has a second, quieter source. Independently of the cursor and of the tidal field, an ambient driver injects velocity across the grid at three superposed timescales: a slow large-scale undulation, a set of wandering pressure zones on an intermediate period, and sparse stress injections drawn from a seeded generator. Amplitudes on all three scale with the current mood. The field runs continuously whether or not anyone is watching, which is the point — the surface has to move on its own before any of its responses to observation can read as choices rather than reflexes. Because the generator is seeded, a given session is reproducible from seed and input history alone; nothing here is frame-to-frame random.

Mood-Driven Material Properties

The Ocean's mood directly changes the physics. When curious, elasticity peaks: restoring forces overshoot the intent target and the surface lifts toward the cursor. When aloof, viscosity rises dramatically, damping both the intent term and the disturbance until the surface flattens and barely registers the contact. When recoiling, viscosity drops and stress propagation rises, so the local disturbance is thrown outward through the neighbour coupling faster than it can build under the cursor — the volatility reads, from above, as withdrawal. These aren't

visual parameters; they're material properties in the C++ solver that change how the physics behaves.

04 — FORMATIONS

Emergent Formations

Six types of structure spawn spontaneously, each with a complete lifecycle — emergence, maturation, dissolution — and each carrying its own spawn weight and lifespan range. The weights are tuned for pacing rather than drawn from the novel, which is inconsistent about how often any of these appear:

Symmetriads — Perfect radial symmetry, 6–12 arms, slow rotation. The longest-lived formation, holding for 30–40 seconds.

Asymmetriads — Towering vertical structures, 50–100m height, irregular shapes that defy easy classification.

Mimoids — Structures the Ocean extrudes to imitate whatever is nearby — a neighboring formation, the suns overhead, the watcher's own outline, or one of a small set of abstract geometric solids — always imperfectly, so the copy never quite becomes the thing it echoes. Tied with extensors for the highest spawn weight.

Vertebrids — Skeletal lattice structures with L-system branching, membrane webbing between ribs, and pearlescent bone-like material. They emerge frozen and are abandoned by the Ocean. The rarest single formation, at the lowest spawn weight in the table.

Extensors — Sprawling horizontal networks with branch-like growth patterns. Frequent but short-lived, dissolving within 10–18 seconds.

Fungoids — Ghostly translucent structures the Ocean extrudes and reabsorbs. Four generative variants (Cap, Filament, Bloom, Columnar), each built from a probability-gated decision tree so every instance is architecturally unique. The variants carry separate weights that sum to the largest share of the table, making fungoids collectively the most frequent thing on the surface even though no single variant is.

JS — FORMATION SPAWN WEIGHTS

```
this.typeConfig = {
  symmetriad:      { weight: 2, lifespan: [30, 40] },
  asymmetriad:     { weight: 2, lifespan: [20, 30] },
  mimoid:          { weight: 3, lifespan: [15, 25] },
  extensor:        { weight: 3, lifespan: [10, 18] },
  vertebrid:       { weight: 1, lifespan: [12, 20] },
  'fungoid-cap':   { weight: 2, lifespan: [18, 30] },
  'fungoid-filament': { weight: 1, lifespan: [15, 25] },
  'fungoid-bloom': { weight: 1, lifespan: [12, 20] },
  'fungoid-columnar': { weight: 1, lifespan: [16, 26] },
};
```

Each formation “breathes” through subtle internal motion, rendered with translucent shaders and flowing 3D noise patterns, suggesting processes we can only glimpse.

Nine lifespan ranges run concurrently, none of them commensurate, and each instance draws from its range independently. Together with the 15–30 second mood interval, the thirty-second memory constant, and the three ambient periods, the result is a set of clocks that never returns to the same configuration twice within a session. No two minutes of the simulation present the same combination of states.

05 — OBSERVATION

Observation Response

The Ocean notices when you're watching. Raycasting detects where the cursor meets the surface, and shader uniforms create localized “awareness” that spreads from that contact point. Hover over the surface and it responds with rippling patterns and increased luminosity.

But the response drifts unpredictably every 15–30 seconds. Sometimes it's curious and the surface leans in under your cursor; sometimes it's aloof and barely registers you; sometimes it's recoiling, pulling back as if the attention itself were unwelcome. You never get told which is coming. This is the sensation the piece is chasing — you're studying it, but it's also studying you.

How to Interact

Movement: WASD or Arrow Keys. When close (< 8 units), walk on the surface. When far (> 8 units), orbit around the planet.

View: Mouse drag to look around.

Zoom: Mouse wheel, or Q/Space to zoom out, E/Shift to zoom in.

Move your cursor slowly to see the Ocean “track” your movements with a delay. The longer you hover in one spot, the more intensely it responds.

06 — INTERFACE

Research Station UI

The simulation includes a CRT-styled research station terminal, toggled with H, that treats the experience as a scientific observation session rather than a game. It's designed to feel like equipment left behind by the researchers in Lem's novel: worn, functional, and slightly glitchy.

A mission brief and logs from earlier observation sessions meet the viewer at the outset, casting them as the latest in a line of researchers — picking up the work rather than starting it.

The terminal's panels reveal incrementally: the globe readout appears after the first formation, a mood bar registering state change after the first transition — value without label — and the psi-contamination index once it crosses a threshold of 0.3. As the Ocean's behavior shifts, the log writes its own entries — a record the viewer never wrote. The psi index itself never resets; prolonged observation leaves a permanent trace.

The aesthetic is intentional. A clean, modern UI would imply comprehension. The CRT terminal implies something older, less certain — instruments built by people who expected to understand more than they did.

07 — ATMOSPHERE

Atmospheric Effects

The simulation includes a multi-layered atmosphere: dynamic exponential fog that responds to sun position, shader-based surface mist with Simplex noise distortion, and periodic plasma eruptions — energy fountains with turbulent curl noise, temperature-based color gradients, and realistic particle physics.

The starfield uses three depth layers with realistic stellar classification (O, B, A, F, G, K, M spectral types) and subtle parallax as the camera moves. Bioluminescence manifests as internal light orbs with pulsing shaders and electric discharge effects beneath the surface.

08 — PERFORMANCE

Performance

The C++ viscoelastic simulation runs in ~1-2ms per frame for the full 128×128 grid (16,384 cells). Each frame, the heightmap and stressmap are read from WASM memory through Float32Array views and bulk-copied into the GPU textures — avoiding any per-element serialization. The rendering pipeline uses a three-tier LOD system that adjusts Ocean surface complexity based on camera distance (128, 64, or 32 segments), instanced geometries for formations and plasma, and shader optimizations including varying caching and precision management.

Results: consistent 60fps on high-end desktops at 1440p, 60fps on mid-range hardware at 1080p, and 30-60fps on modern mobile devices with automatic quality adjustment. The WASM simulation gracefully degrades — if it fails to load, the Ocean falls back to shader-only rendering.

09 — PHILOSOPHY

Design Philosophy

Solaris resists easy comprehension by design. There are no objectives, no progression systems, no tutorials. The Ocean operates on its own timeline — formations emerge and dissolve without viewer input, mood states shift without announcement, responses vary without explanation. The goal is not mastery but sustained attention: a space where observation is possible but understanding is not.

Related Publications

- Hamilton, Magda. 2026. *Solaris: Designing for Unknowability in Interactive Systems*. In *Proceedings of the Conference on Animation and Interactive Art (Expanded '26)*. Association for Computing Machinery, New York, NY, USA. Forthcoming.

Develops the design argument behind this work — why it's built to resist interpretation, and where that approach succeeds and fails.

Bibliography

- [1] Lem, S. (1961). *Solaris*. Warsaw: Wydawnictwo MON.
- [2] Newton, I. (1687). *Philosophiæ Naturalis Principia Mathematica*. London.
- [3] Poincaré, H. (1892–1899). *Les Méthodes Nouvelles de la Mécanique Céleste* (3 vols.). Paris: Gauthier-Villars. English edition: *New Methods of Celestial Mechanics*, ed. D. L. Goroff. American Institute of Physics, 1993.
- [4] Barrow-Green, J. (1997). *Poincaré and the Three Body Problem*. Providence, RI: American Mathematical Society.

How to cite this work

Hamilton, Magda. 2026. *Solaris: A WebAssembly/WebGL Architecture for Real-Time Viscoelastic Continuum Simulation and Tidal-Force Visualization in the Browser*. Technical Report, Velvet Disruption. DOI 10.5281/zenodo.21540788.

© 2026 Magda Hamilton. Distributed under a Creative Commons Attribution 4.0 International (CC BY 4.0) licence.

First published 22 January 2026 at magdahamilton.dev/solaris.html; the C++/WebAssembly architecture documented here was added on 16 February 2026. The second edition was prepared 22 July 2026; this third edition, 24 July 2026.